

Serie ASW
Applicazioni e Servizi Web

Viola Leonardo - 0001199015 {leonardo.viola2@studio.unibo.it}

12 febbraio 2026

1 Introduzione

Il progetto consiste nello sviluppo di una web app dinamica dedicata alla gestione e al monitoraggio di un campionato di calcio, che permetta di visualizzare le principali informazioni di un campionato, quindi consultare il suo calendario, vedere lo storico delle partite, la classifica, i dettagli delle singole partite, le informazioni delle singole squadre e dei relativi giocatori, le statistiche sia individuali dei giocatori che delle squadre.

Ci sono due tipologie di utenti del sistema:

- **Utente Normale:** ovvero l'appassionato di calcio che visita il sito principalmente per vedere il risultato in tempo reale delle partite, e per consultare la classifica e le statistiche del campionato, ma potrebbe anche voler visualizzare le informazioni relative alle squadre o ai giocatori.
- **Utente Admin:** ovvero colui che si occupa di gestire le partite in tempo reale mantenendo il sistema aggiornato e coerente con quello che accade nella realtà. Quindi man mano che nella realtà la partita inizia e va avanti con il suo svolgimento questo utente aggiorna il sistema in modo che gli utenti normali che stanno seguendo la partita dalla web app possano vedere in tempo reale gli aggiornamenti della partita, quindi ad esempio si occupa di avviare la partita, inserire le diverse tipologie di eventi (goal, falli, calci d'angolo, ecc...), far terminare la partita.

2 Requisiti

Il sistema è stato progettato per soddisfare i seguenti requisiti funzionali suddivisi per tipologia di utente:

Requisiti per utente normale:

- Poter visualizzare la classifica del campionato
- Poter visualizzare le squadre partecipanti al campionato con i relativi dettagli
- Poter visualizzare i giocatori di una squadra con i relativi dettagli
- Poter visualizzare il calendario del campionato, con lo storico delle partite già giocate e le partite programmate
- Poter visualizzare i dettagli della partita, formazioni e statistiche, e cronaca ovvero lista di eventi (goal, falli, cartellini, ecc...)
- **Sincronizzazione Live:** Aggiornamento del minuto di gioco, dello stato e della cronaca (ovvero la lista di eventi) della partita senza necessità di ricaricare la pagina.

Requisiti per utente admin:

- Poter effettuare login nel sistema come utente admin
- Poter avviare una partita non ancora iniziata e gestire l'andamento della partita inserendo eventi, un evento potrebbe essere associato ad un calciatore (es: goal), oppure ad una squadra in generale (es: calcio d'angolo), ed inoltre deve poter cambiare lo stato della partita, terminando il primo tempo di gioco ed avviando poi il secondo tempo in cui sarà possibile inserire nuovi eventi fino al termine della partita.
- La partita non terminerà automaticamente allo scoccare del novantesimo minuto, così come il primo tempo non terminerà in automatico allo scoccare del quarantacinquesimo minuto, ma sarà l'admin a terminare il tempo di gioco questo per gestire eventuali minuti di recupero.
- L'admin non può modificare in nessun caso il risultato e gli eventi di una partita terminata.
- Non deve essere possibile per una squadra effettuare più sostituzioni di quelle previste (ovvero 5).

Come requisito opzionale che per mancanza di tempo non sono riuscito ad implementare c'era l'elezione del MVP, da parte di un modello di AI che usando le statistiche dei giocatori in partita decretava il miglior giocatore.

Trattandosi di partite di campionato non è previsto il caso in cui una partita finisca ai tempi supplementari o ai calci di rigore, inoltre si assume che le partite siano già presenti nel database con tutte le informazioni necessarie, data ed ora di inizio, formazioni delle due squadre, ecc... .

3 Design

3.1 Design architetturale

L'architettura del sistema può essere suddivisa tra backend e frontend, entrambi suddivisi in più moduli. Il backend si occupa di mettere a disposizione del frontend delle "interfacce", chiamate **routes** che permettono al frontend di leggere oppure scrivere dei dati sul database. Mentre il frontend si occupa di presentare questi dati all'utente finale.

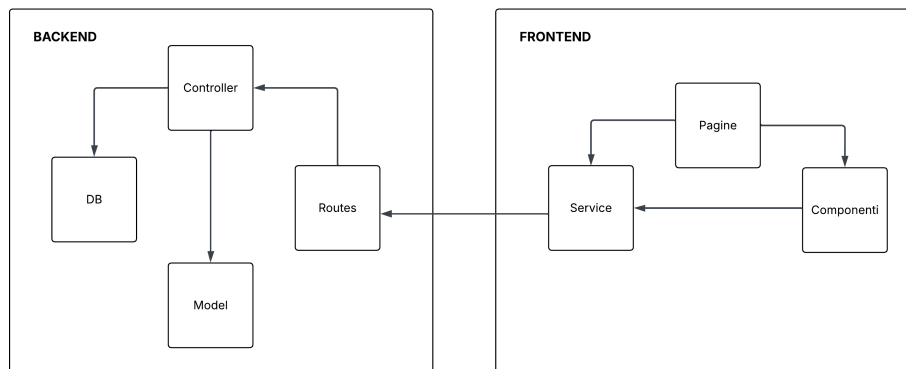


Figure 1: Design dell'architettura

3.1.1 Backend

Il **Model** del sistema è quella parte di sistema che serve a rappresentare le entità del dominio applicativo modellato, le entità del model sono:

- **Match:** modella la partita. Ogni partita si trova in uno dei seguenti stati:
 1. *NON_INIZIATA*,
 2. *IN_CORSO_PRIMO_TEMPO*,
 3. *FINE_PRIMO_TEMPO*,
 4. *IN_CORSO_SECONDO_TEMPO*,
 5. *FINITA*,
 6. *SOSPESA*,
 7. *ANNULLATA*

Inoltre la partita oltre ad altri dati come la data ed ora di inizio, le squadre, ecc..., contiene la lista di eventi che accadono durante la partita, ogni evento può appartenere ad una delle seguenti tipologie:

1. *GOAL*,

2. *AMMONIZIONE*,
3. *ESPULSIONE*,
4. *FALLO*,
5. *RIGORE*,
6. *ANGOLO*,
7. *SOSTITUZIONE*

- **Player:** modella il giocatore, contiene dati anagrafici del giocatore.
- **Referee:** modella l'arbitro, contiene i dati anagrafici dell'arbitro.
- **Team:** modella la squadra, contiene i dati del team come città, logo, link social.
- **Standing:** modella la classifica del campionato, contiene per ogni squadra i punti, i goal fatti, il numero di partite giocate, ecc...
- **PlayerStats:** modella le statistiche di un giocatore, per ogni giocatore contiene diverse statistiche come il numero di goal, di falli, di cartellini, ecc... .
- **User:** modella l'utente del sistema.

Il modulo del **Controller** è quella parte di sistema che mette in contatto il backend con il database usando gli schemi definiti nel model, in modo da poter leggere, scrivere, o eliminare dati. Ho suddiviso logicamente questo modulo creando un controller per ogni entità del model, in questo modo in ogni controller ho i metodi relativi ad una singola entità del model.

Le **Routes** invece sono il punto di contatto che il frontend usa per interfacciarsi con il backend, servono quindi a mettere in comunicazione il frontend con il database, infatti ad ogni route è associato un metodo del controller che compie una determinata azione sul database, che può essere di lettura, di modifica o di eliminazione.

Le routes sono state suddivise in base alle entità del dominio. Di seguito sono elencati gli endpoint disponibili, composti dal prefisso definito nel router principale e dai percorsi specifici.

Autenticazione

- **POST** */auth/login*: verifica username e password

Squadre

- **GET** */teams/*: Recupera la lista di tutte le squadre partecipanti.
- **GET** */teams/:id*: Recupera i dettagli di una singola squadra tramite ID.
- **GET** */teams/:teamId/players*: Recupera la lista di tutti i calciatori appartenenti a una specifica squadra.

Partite

- **GET** */matches/*: Recupera tutte le partite.
- **GET** */matches/giornata/:giornata*: Filtra le partite in base alla giornata di campionato selezionata.
- **GET** */matches/squadra/:teamId*: Recupera tutte le partite giocate o programmate di una specifica squadra.
- **GET** */matches/:id*: Recupera i dettagli completi, formazioni ed eventi di una singola partita.

Azioni Live

- **POST** */matches/:id/start-first-half*: Avvia il primo tempo della partita.
- **POST** */matches/:id/start-second-half*: Avvia il secondo tempo della partita.
- **POST** */matches/:id/end-period*: Termina il tempo corrente (intervallo o fischio finale).
- **POST** */matches/:id/events*: Registra un nuovo evento live (Goal, Ammonizione, ecc.) nel database.

Giocatori

- **GET** */players/*: Recupera l'anagrafica di tutti i giocatori del campionato.
- **GET** */players/:id*: Recupera i dettagli anagrafici di un singolo giocatore.

Classifiche

- **GET** */standings/*: Recupera la classifica aggiornata del campionato.

Statistiche

- **GET** */stats/teams/full*: Recupera le statistiche aggregate di tutte le squadre (vittorie, sconfitte, gol fatti/subiti).
- **GET** */stats/players/:id*: Recupera le statistiche individuali di un calciatore (gol totali, assist, cartellini).
- **GET** */stats/top-scorers*: Restituisce la classifica dei marcatori.
- **GET** */stats/top-assists*: Restituisce la classifica degli assistman.
- **GET** */stats/top-yellows*: Restituisce i giocatori con più ammonizioni.
- **GET** */stats/top-reds*: Restituisce i giocatori con più espulsioni.
- **GET** */stats/teams/:teamId*: Recupera le statistiche di tutti i giocatori appartenenti a una specifica squadra.

3.1.2 Frontend

Il modulo del **Service** è quella parte di frontend che viene usato per interfacciarsi con il backend, quindi si occupa di andare a richiamare le routes del backend, in modo da recuperare dati da mostrare all'utente, oppure passare dati ricevuti dall'utente.

Le **Pages** Sono gli elementi che rappresentano le pagine della web app, ogni pagina a sua volta può usare dei componenti definiti in altri files e che a loro volta possono essere utilizzati anche in altre pagine. In questo modo si limitano le ripetizioni di codice, aumentando la manutenibilità, perché una singola modifica al componente si ripercuote in automatico su tutte le pagine che usano quel componente.

I **Components** sono i singoli blocchi che formano le pagine, posso avere anche un componente che usa a sua volta altri componenti. Un componente può ricevere dei dati dal componente padre, che come detto può essere una pagina, oppure un altro componente.

3.2 Design delle interfacce

Per la progettazione dell'interfaccia ho preso spunto da diversi siti ed integrato diverse idee nel modo che ritenevo migliore. Sono partito sviluppando dei mockups che sono stati molto utili nello sviluppo della struttura delle pagine, durante lo sviluppo dei mockups non mi sono concentrato tanto sull'estetica, quindi colori di sfondo, bordi, ecc..., ma sono stati realizzati con lo scopo di capire l'architettura della pagina, e quindi in quali componenti suddividere una pagina, dove poter riusare gli stessi componenti e le vie di interazione dell'utente con la web app.

Ecco i mockups di alcune pagine:

- Mockup della pagina home che raffigura in alto la lista di ultime partite giocate, al centro la classifica parziale, ed in fondo la lista delle squadre

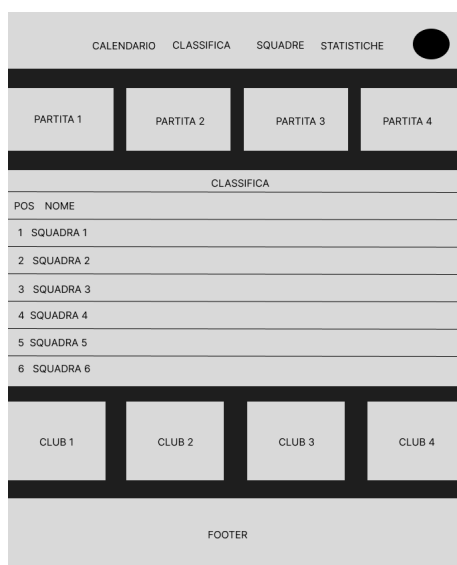


Figure 2: Mockup della pagina home

- Mockup della pagina di dettaglio della partita che mostra in alto il risultato, e sotto 3 tabs per poter vedere tutte le informazioni della partita



Figure 3: Mockup della pagina della partita

- Mockup della pagina della classifica che mostra diverse informazioni come differenza reti, partite giocate, partite perse, vinte e pareggiate, ecc... .

CALENDARIO CLASSIFICA SQUADRE STATISTICHE										
CLASSIFICA										
POS	NOME	PG	V	N	P	GF	GS	DR	PT	
1	SQUADRA 1									
2	SQUADRA 2									
3	SQUADRA 3									
4	SQUADRA 4									
5	SQUADRA 5									
6	SQUADRA 6									
7	SQUADRA 7									
8	SQUADRA 8									
9	SQUADRA 9									
10	SQUADRA 10									
FOOTER										

Figure 4: Mockup della pagina della classifica

- Mockup della pagina di dettaglio della squadra, in alto è presente un header con alcune informazioni, e sotto 4 tabs in cui saranno presenti informazioni più dettagliate, tra cui anche la lista di giocatori
- Mockup della pagina di calendario, in cui è possibile tramite il button in alto selezionare la giornata di campionato che si vuole visualizzare, ed

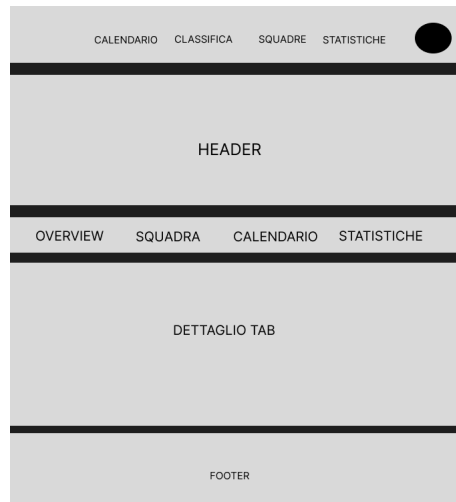


Figure 5: Mockup della pagina di dettaglio del club

in basso sono presenti le cards delle partite della giornata di campionato selezionata.



Figure 6: Mockup della pagina del calendario

- Mockup della pagina con la lista di tutte le squadre del campionato

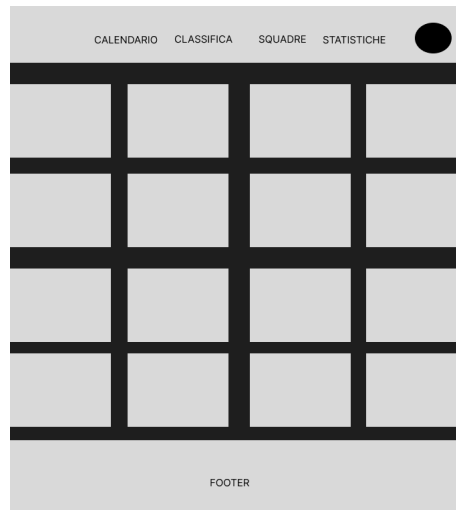


Figure 7: Mockup della pagina delle squadre

I mockup sono stati utili anche per individuare quelle che erano le principali modalità di interazione dell'utente, grazie a diversi test e prove si è riusciti a migliorare quelli che erano le interazioni dell'utente con le diverse pagine riducendo il numero di click arrivare alle pagine di maggior interesse per l'utente.

Nella figura 8 sono riportate le principali modalità di interazione tra le diverse pagine.

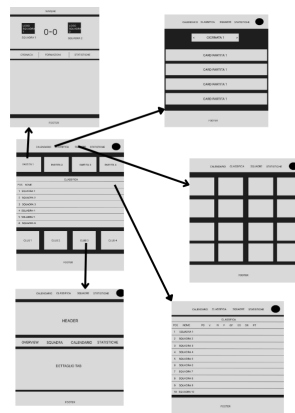


Figure 8: Figura che mostra le principali interazioni tra le diverse pagine.

4 Tecnologie

Per lo sviluppo è stato adottato lo stack **MEVN**, scelto per la sua efficienza nella gestione di dati JSON-like e per la reattività dell'interfaccia:

- **MongoDB:** Database NoSQL scelto per la flessibilità dello schema (fondamentale per gestire array di eventi di tipo diverso all'interno dei match).
- **Express.js:** Framework lato server per la gestione delle rotte API e della logica di business.
- **Vue.js 3 (Composition API):** Framework frontend utilizzato per la gestione dello stato reattivo e dei componenti modulari (come visto nella gestione della Timeline).
- **Node.js:** Il Runtime Environment. Il motore che permette di eseguire JavaScript sul server, gestendo le richieste HTTP e il traffico dati.

Altre tecnologie utilizzate sono:

- **Mongoose:** Libreria ODM (Object Data Modeling) per MongoDB e Node.js. È stata utilizzata per definire gli schemi del model e per facilitare l'interazione con il database tramite un'interfaccia basata su oggetti.
- **Socket.io:** Libreria che abilita la comunicazione in tempo reale tra client e server. È il cuore della *Sincronizzazione Live*, permettendo agli utenti di ricevere aggiornamenti sui match (come il cambio di stato o del risultato) senza ricaricare la pagina.
- **Axios:** Client HTTP basato su promise utilizzato nel frontend per effettuare richieste alle API del backend in modo semplice e gestirne le risposte.
- **Vite:** Build tool di nuova generazione scelto per ottimizzare le performance durante lo sviluppo e per garantire una compilazione rapida del codice frontend in fase di produzione.
- **Chart.js:** Libreria JavaScript utilizzata per la visualizzazione dinamica dei dati. È stata impiegata per generare grafici a torta e istogrammi relativi alle statistiche dei match e dei singoli giocatori, rendendo l'analisi delle performance sportive immediata e visivamente accattivante.
- **CORS (Cross-Origin Resource Sharing):** Middleware per Express fondamentale per gestire la sicurezza delle comunicazioni. È stato configurato per autorizzare le richieste provenienti dal dominio del frontend (Vite), superando le restrizioni della *Same-Origin Policy* e permettendo lo scambio di dati e la connessione Socket.io.
- **Bcrypt:** Libreria utilizzata per l'hashing sicuro delle password. Usato nello script di inizializzazione del sistema, per cifrare le credenziali dell'utente admin.

- **Pinia:** Lo store manager ufficiale per Vue.js 3, utilizzato per la gestione dello stato globale dell'applicazione. È stato usato per centralizzare le informazioni sull'autenticazione dell'utente e per condividere i dati tra componenti distanti tra loro.
- **JSON Web Token (JWT):** utilizzato per la creazione di token di accesso sicuri. Nel progetto, JWT è impiegato per gestire l'autenticazione *stateless*: al login, il server genera un token firmato digitalmente che il client memorizza e invia nelle successive richieste HTTP. Questo meccanismo permette al backend di verificare l'identità e i permessi dell'amministratore senza dover consultare continuamente il database, migliorando sicurezza e prestazioni.

5 Codice

Un aspetto rilevante del codice riguarda la gestione della logica degli eventi live. Se l'admin ha effettuato il login allora nella pagina della partita compare il componente *MatchAdminControls*, ovvero il panel usato dall'admin per gestire la partita. Nel caso la partita sia già terminata l'admin non può fare nulla, quindi il panel sarà vuoto, ma nel caso la partita deve ancora iniziare sarà presente il button *"Inizia Partita"* che modifica lo stato e salva il timestamp di inizio della partita.

5.1 Gestione del minuto di gioco

Il timestamp salvato all'avvio della partita è molto importante ed è legato al minuto di gioco della partita, all'inizio ero indeciso su come implementare questa parte in modo da garantire che il minuto di gioco si aggiorni dinamicamente così da dare l'idea che la partita sia live e che il minuto di gioco stia andando avanti senza che sia l'utente a dover fare il refresh della pagina di continuo.

Per evitare che fosse il backend in continuazione a fare il push del nuovo minuto di gioco per notificare ai clients che il minuto di gioco era cambiato ho deciso di salvare al momento dell'inizio della partita il timestamp, in questo modo il client in autonomia può impostare un timer che ogni tot di secondi riesegue una funzione che calcola il minuto di gioco partendo dal timestamp di inizio partita.

In questo modo evito di avere un flusso costante di notifiche dal backend al frontend per notificare ogni volta il cambio di minuto.

Ovviamente al momento dell'inizio della partita salvo un timestamp, ma anche all'inizio del secondo tempo salvo un altro timestamp, in modo da poter replicare anche nel secondo tempo il calcolo corretto del minuto.

Ho aggiunto al calcolo del tempo una costante di accelerazione di 0.3, in modo che la partita duri meno di novanta minuti, in particolare ogni 5 secondi il minuto della partita aumenta.

Listing 1: Timer che ricalcola il minuto di gioco ogni secondo se la partita è in corso.

```
1 // Funzione che prende come parametro l'oggetto partita e
  restituisce il minuto di gioco in tempo reale
2 const calculateLiveMinute = (data) => {
3   if (!data) return 0;
4   let riferimento = null;
5   let offset = 0;
6
7   // Determina il riferimento temporale in base allo stato
    della partita
8   // recuperando i timestamp di inizio tempi
9   if (data.stato === 'IN_CORSO_PRIMO_TEMPO') {
10     riferimento = data.inizioPrimoTempo;
11   } else if (data.stato === 'IN_CORSO_SECONDO_TEMPO') {
```

```

12     riferimento = data.inizioSecondoTempo;
13     offset = 45;
14 }
15
16 if (riferimento) {
17     const start = new Date(riferimento).getTime();
18     const now = new Date().getTime();
19     // Differenza in secondi tra orario attuale e orario
20     // di inizio partita
21     const diffSeconds = (now - start) / 1000;
22     // Fattore di velocita' per simulare il tempo di gioco
23     // reale
24     const factor = 0.3;
25
26     // Calcolo del minuto di gioco
27     let min = Math.floor(diffSeconds * factor) + 1 +
28         offset;
29
30     // Limita il minuto al massimo a 45 o 90 minuti
31     return Math.min(min, offset === 0 ? 45 : 90);
32 } else {
33     // Se la partita non e' in corso, restituisco il
34     // minuto finale del tempo
35     if (data.stato === 'FINE_PRIMO_TEMPO') return 45;
36     if (data.stato === 'FINITA') return 90;
37     return 0;
38 }
39 };
40
41 onMounted(() => {
42     ...
43     timerInterval = setInterval(() => {
44         if (match.value && match.value.stato.startsWith('
45             IN_CORSO')) {
46                 match.value.minutoCorrente = calculateLiveMinute(
47                     match.value);
48             }
49         }, 1000);
50     ....
51 });

```

5.2 Gestione inserimento di un evento

Per gestire le notifiche relative all'inserimento di nuovi eventi ho usato la libreria **Socket.io** che mi ha permesso di avere una comunicazione tra backend e frontend basata su eventi.

Sia il frontend che il backend possono comunicarsi l'inserimento di un nuovo

evento, l'admin inserisce un nuovo evento dal frontend comunicandolo al backend, e a sua volta il backend invia la notifica di nuovo evento a tutti i clients collegati.

5.2.1 Invio evento al backend

Lato frontend per comunicare al backend che l'inserimento di un nuovo evento viene richiamata la funzione del service che richiama la route del backend per l'inserimento di un nuovo evento.

Listing 2: Funzione usata per richiamare route del backend per inserimento nuovo evento.

```
1  /**
2   * Aggiunge un evento alla partita
3   */
4  export async function addLiveEvent(id, eventData) {
5    const res = await api.post(`/matches/${id}/events`,
6      eventData);
7    return res.data;
8  }
```

5.2.2 Gestione inserimento evento lato backend

Il metodo del controller che si occupa di gestire l'inserimento di un evento nel db e conseguentemente inviare la notifica ai clients collegati tramite il socket è il seguente

Listing 3: Funzione lato backend che inserisce evento nel database ed emette evento sul socket.

```
1  /**
2   * POST /matches/:id/events
3   */
4  exports.addLiveEvent = async (req, res) => {
5    try {
6      // cerco la partita tramite matchId
7      const match = await Match.findOne({ matchId: Number(req.
8        params.id) });
9      if (!match) return res.status(404).json({ message:
10        'Match non trovato' });
11
12      // estraggo i dati del nuovo evento
13      const { tipo, squadraId, playerId, playerOutId, minuto,
14        dettaglio } = req.body;
15
16      // costruisco il nuovo evento
17      const nuovoEvento = {
18        tipo,
19        matchId: match.matchId,
20        playerId,
21        playerOutId,
22        minuto,
23        dettaglio
24      };
25      await Match.updateOne({ matchId: match.matchId },
26        { $push: { events: nuovoEvento } }, { upsert: true });
27      socket.emit('nuovoEvento', nuovoEvento);
28    } catch (error) {
29      console.log(error);
30    }
31  }
```



```

16     squadraId: Number(squadraId),
17     minuto: Math.min(minuto, 90),
18     dettaglio,
19     playerOutId: playerOutId ? Number(playerOutId) : null,
20     playerId: playerId ? Number(playerId) : null
21 };
22
23 // --- EVENTO GOAL ---
24 // se si tratta di un evento di tipo GOAL aggiorno il
    risultato del match
25 if (tipo === GOAL ) {
26     if (Number(squadraId) === match.squadre.casa.teamId)
27         match.risultato.casa++;
28     else
29         match.risultato.trasferta++;
30 }
31
32 match.eventi.push(nuovoEvento);
33 await match.save();
34
35 // --- PUSH REAL-TIME ---
36 const io = req.app.get('io');
37 if (io) {
38     io.emit('matchUpdate', {
39         matchId: match.matchId,
40         nuovoEvento: nuovoEvento,
41         risultato: match.risultato
42     });
43 }
44
45 res.status(201).json(match);
46 } catch (err) {
47     console.error( Errore addLiveEvent: , err);
48     res.status(500).json({ error: err.message });
49 }
50 };

```

I possibili eventi che possono essere inviati sul socket sono due:

- **matchStatusUpdate**
- **matchUpdate**

in questo caso dato che si tratta di un inserimento di evento e non di un cambio di stato della partita l'evento che lancia sul socket è quello di **matchUpdate**.

5.2.3 Ricezione notifica inserimento evento lato frontend

Lato frontend nell'*onMounted* della pagina della partita mi metto in ascolto dell'evento *matchUpdate* e alla sua ricezione aggiungo l'evento ricevuto (ovvero l'evento appena inserito dall'admin) alla lista di eventi dell'oggetto *match*.

Listing 4: Listener lato frontend dell'evento 'matchUpdate'.

```
1  onMounted(() => {
2    ...
3    socket.on('matchUpdate', (data) => {
4      if (match.value && data.matchId === match.value.
5        matchId) {
6        // Aggiorna punteggio e aggiunge l'evento alla lista
7        match.value.risultato = data.risultato;
8        if (!match.value.eventi) match.value.eventi = [];
9        match.value.eventi.push(data.nuovoEvento);
10     }
11   });
12   ...
13   });
```

5.3 Gestione cambio di stato della partita

Il cambio di stato della partita può significare che la partita è iniziata, che è terminato il primo tempo, che è iniziato il secondo tempo oppure che è terminata la partita. Anche per gestire il cambio di stato ho usato la stessa modalità di notifica usata per l'inserimento di eventi, quindi per inviare il nuovo evento dal frontend al backend richiamo la route, il backend gestisce questa richiesta salvando sul db il nuovo stato ed emette un evento sul socket che verrà ricevuto da tutti i clients in ascolto.

5.3.1 Invio cambio di stato al backend

Nel pannello dell'admin oltre al button per inserire l'evento è presente il button che ti permette di iniziare o terminare il tempo di gioco a seconda dello stato corrente della partita, e che quindi richiamerà i seguenti metodi del service che appunto richiamano le routes del backend:

Listing 5: Funzioni del frontend che richiamano routes del backend per inizio e fine dei tempi di gioco.

```
1  /**
2   * Inizia il primo tempo (imposta stato e timestamp)
3   */
4  export async function startFirstHalf(id) {
5    const res = await api.post(`/matches/${id}/start-first-
6      half`);
7    return res.data;
8  }
9
10 /**
11  * Inizia il secondo tempo (resetta timestamp)
12  */
13 export async function startSecondHalf(id) {
```

```

13   const res = await api.post(`/matches/${id}/start-second-
      half`);
14   return res.data;
15 }
16
17 /**
18  * Termina il tempo attuale (Intervallo o Fine Gara)
19  */
20 export async function endPeriod(id) {
21   const res = await api.post(`/matches/${id}/end-period`);
22   return res.data;
23 }

```

5.3.2 Gestione cambio di stato lato backend

All'avvio della partita come si vede dal metodo *startFirstHalf* riportato di seguito modifico lo stato, salvo il timestamp di inizio partita e salvo poi le modifiche sul database. Dopo aver salvato invio l'evento *matchStatusUpdate* sul socket per comunicare ai clients che c'è stato un cambio di stato.

Listing 6: Funzione del backend che gestisce l'inizio della partita.

```

1  /**
2   * POST /matches/:id/start-first-half
3   * Inizia il primo tempo di una partita
4   */
5  exports.startFirstHalf = async (req, res) => {
6    try {
7      // ricerco la partita tramite matchId
8      const match = await Match.findOne({ matchId: Number(req.
          params.id) });
9      if (!match) return res.status(404).json({ message:
          Match non trovato });
10
11      // se il match e' gia' iniziato non posso far partire il
          primo tempo
12      if (match.stato !== NON_INIZIATA ) {
13        return res.status(400).json({ message: Il match e'
          gi iniziato o concluso });
14      }
15
16      // aggiorno stato e salvo timestamp
17      match.stato = IN_CORSO_PRIMO_TEMPO ;
18      match.inizioPrimoTempo = new Date();
19
20      // salvo le modifiche
21      await match.save();
22
23      // --- PUSH REAL-TIME ---

```

```

24     const io = req.app.get('io');
25     if (io) {
26         io.emit('matchStatusUpdate', {
27             matchId: match.matchId,
28             stato: match.stato,
29             inizioPrimoTempo: match.inizioPrimoTempo,
30             message:   Fischio d'inizio!
31         });
32     }
33
34     res.json(match);
35 } catch (err) {
36     res.status(500).json({ error: err.message });
37 }
38 };

```

In maniera analoga funziona il metodo *startSecondHalf* che serve per avviare il secondo tempo, quindi modifco lo stato, salvo il timestamp di inizio secondo tempo, salvo sul database e poi notifico sul socket.

Listing 7: Funzione del backend che gestisce l'inizio del secondo tempo.

```

1  /**
2   * POST /matches/:id/start-second-half
3   */
4  exports.startSecondHalf = async (req, res) => {
5      try {
6          // ricerco la partita tramite matchId
7          const match = await Match.findOne({ matchId: Number(req.
8              params.id) });
9          if (!match) return res.status(404).json({ message:
10              Match non trovato });
11
12          if (match.stato !== FINE_PRIMO_TEMPO ) {
13              return res.status(400).json({ message: Stato non
14                  valido per inizio secondo tempo });
15          }
16
17          match.stato = IN_CORSO_SECONDO_TEMPO ;
18          match.inizioSecondoTempo = new Date();
19
20          await match.save();
21
22          // --- PUSH REAL-TIME ---
23          const io = req.app.get('io');
24          if (io) {
25              io.emit('matchStatusUpdate', {

```

```

26     message:  Inizio secondo tempo!
27   });
28 }
29
30   res.json(match);
31 } catch (err) {
32   res.status(500).json({ error: err.message });
33 }
34 };

```

Per quanto riguarda la terminazione del tempo di gioco viene usata la funzione *endPeriod* che in serve a terminare il tempo di gioco, e quindi in base allo stato corrente della partita imposta lo stato su *FINE_PRIMO_TEMPO* o *FINITA*.

Nel caso la partita sia finita esegue l'aggiornamento della classifica (basandosi sul risultato finale) e delle statistiche dei giocatori (basandosi sugli eventi della partita) tramite le funzioni *internalUpdateStandings(match)* e *internalUpdatePlayerStats(match)*, ed infine salva.

Ovviamente anche in questo caso dopo il salvataggio notifica ai clients emettendosul socket l'evento *matchStatusUpdate*.

Listing 8: Funzione del backend che gestisce l'inizio della partita.

```

1  /**
2   * POST /matches/:id/end-period
3   * Termina il tempo di gioco, se e' la fine del secondo
4     tempo, aggiorna tutto automaticamente.
5   */
6  exports.endPeriod = async (req, res) => {
7    try {
8      // ricerco la partita tramite matchId
9      const match = await Match.findOne({ matchId: Number(req.
10        params.id) });
11      if (!match) return res.status(404).json({ message:
12        Match non trovato });
13
14      let isGameOver = false;
15
16      // Aggiorno lo stato della partita e verifico se
17        finita
18      if (match.stato === IN_CORSO_PRIMO_TEMPO ) {
19        match.stato = FINE_PRIMO_TEMPO ;
20      } else if (match.stato === IN_CORSO_SECONDO_TEMPO ) {
21        match.stato = FINITA ;
22        isGameOver = true;
23      } else {
24        return res.status(400).json({ message: Match non in
25          corso });
26      }
27    }
28  }

```

```

23 // Se la partita e' finita, eseguo gli aggiornamenti
    alla classifica e statistiche giocatori
24 if (isGameOver) {
25     await internalUpdateStandings(match);
26     await internalUpdatePlayerStats(match);
27 }
28
29 await match.save();
30
31 // --- PUSH REAL-TIME ---
32 const io = req.app.get('io');
33 if (io) {
34     io.emit('matchStatusUpdate', {
35         matchId: match.matchId,
36         stato: match.stato,
37         message: isGameOver ? Partita finita! Classifica
            aggiornata. : Fine primo tempo
38     });
39 }
40
41 res.json({
42     message: isGameOver ? Partita conclusa e dati
        aggiornati : Periodo terminato ,
43     match
44 });
45 } catch (err) {
46     console.error( Errore durante endPeriod: , err);
47     res.status(500).json({ error: err.message });
48 }
49 };

```

5.3.3 Ricezione notifica cambio di stato lato frontend

Anche in questo caso come per l'evento *matchUpdate* nell'*onMounted()* della pagina della partita mi metto in ascolto dell'evento *matchStatusUpdate* sul socket

Listing 9: Listener dell'evento 'matchStatusUpdate'.

```

1 onMounted(() => {
2     ...
3     // Gestione dei cambi di stato
4     socket.on('matchStatusUpdate', (updatedData) => {
5         if (match.value && updatedData.matchId === match.value
            .matchId) {
6             // Aggiorno i dati core
7             match.value.stato = updatedData.stato;
8
9             // Aggiorno i timestamp se presenti

```

```

10     if (updatedData.inizioPrimoTempo) {
11         match.value.inizioPrimoTempo = updatedData.
            inizioPrimoTempo;
12     }
13     if (updatedData.inizioSecondoTempo) {
14         match.value.inizioSecondoTempo = updatedData.
            inizioSecondoTempo;
15     }
16
17     // Ricalcolo immediato del minuto
18     match.value.minutoCorrente = calculateLiveMinute(
        match.value);
19     }
20     });
21     ...
22 });

```

5.4 Gestione del login

Per evitare che qualsiasi utente richiamando le routes del backend usate per modificare lo stato della partita ed inserire eventi tramite un qualsiasi tool per effettuare delle richieste POST come Postman, ho inserito un controllo nel backend che verifica se la richiesta è accompagnata da un token, quindi in tutti i metodi del controller usati per modificare lo stato della partita o inserire eventi ho aggiunto il seguente controllo:

Listing 10: Codice usato nei metodi del controller per verificare che la richiesta abbia un token e che il token sia valido.

```

1  const authHeader = req.headers['authorization'];
2  // Prende il token dopo Bearer
3  const token = authHeader && authHeader.split(' ')[1];
4
5  if (!token) {
6      return res.status(401).json({ message: Token mancante:
        autenticazione richiesta });
7  }
8
9  // Verifico che il token sia vero e non falsato usando la
    chiave segreta
10 try {
11     const decoded = jwt.verify(token, JWT_SECRET);
12
13     // Verifico esplicitamente se un administrator
14     if (decoded.role !== 'administrator') {
15         return res.status(403).json({ message: Richiesti
        privilegi di amministratore });
16     }
17 } catch (err) {

```

```

18   return res.status(403).json({ message: Token non valido o
19   scaduto });
  }

```

Lato frontend invece ho usato un interceptor che intercetta la chiamata che il frontend sta per fare ed inietta nella richiesta il token se presente:

Listing 11: Interceptor delle richieste effettuate al server.

```

1  api.interceptors.request.use( (config) => {
2    const auth = useAuthStore()
3    // Prende il token dallo store Pinia
4    const token = auth.token
5
6    // se il token esiste lo inserisce nell'header
7    // Authorization
8    if (token) {
9      // Inserisce il token nel formato standard Bearer
10     config.headers.Authorization = `Bearer ${token}`
11   }
12   return config
13 },
14 (error) => {
15   return Promise.reject(error)
16 }
17 )

```

il token lo recupera da uno store pinia, infatti al momento del login il server risponde inviando i dati dell'utente ed il token, token che viene creato a sua volta inserendo al suo interno anche id e ruolo utente

Listing 12: Creazione token ed invio al frontend.

```

1  // CREAZIONE DEL TOKEN
2  // Inserisco nel token l'ID e il ruolo dell'utente
3  const token = jwt.sign(
4    { id: user._id, role: user.role },
5    JWT_SECRET,
6    { expiresIn: '2h' }
7  );
8
9  // INVIO DEL TOKEN AL FRONTEND
10 res.json({
11   success: true,
12   token: token,
13   user: {
14     id: user._id,
15     username: user.username,
16     role: user.role
17   }
18 });

```


il frontend riceve questa risposta (in caso username e password siano corretti) e salva i dati in uno store in modo che tutte le pagine ed i componenti possano vedere che l'admin ha effettuato il login.

6 Test

Le funzionalità del sistema sono state testate sia su browser Chrome che su browser Edge con l'obiettivo di garantire portabilità. Inoltre, sono stati effettuati test anche su dispositivo mobile per garantire che la web app fosse responsive. In ogni caso i test hanno voluto verificare che le funzionalità principali funzionassero allo stesso modo in tutti i browser e dispositivi e che la visualizzazione fosse corretta anche su piattaforme differenti.

Una volta terminato, il sistema è stato sottoposto all'attenzione di entrambe le tipologie di utente coinvolte in modo da valutare la correttezza dell'uso del sistema. In questa ultima fase mi sono immedesimato in entrambe le tipologie di utente, testando opportunamente il sistema.

Anche le API sviluppate lato server sono state opportunamente testate utilizzando lo strumento Postman che ha permesso di verificare che il loro comportamento fosse quello desiderato, prima di procedere con l'integrazione con il front-end e quindi con il test finale relativamente alla funzionalità completa dell'applicazione.

7 Deployment

7.1 Installazione

Per installare l'applicativo è necessario seguire le istruzioni sottoelencate:

1. Clonazione del repository con comando `'git clone https://github.com/leonardo284/ProgettoAWS.git'`
2. Tramite una Shell Bash spostarsi all'interno della cartella dell'elaborato
3. eseguire `'cd .'`
4. eseguire `'npm install'`
5. eseguire `'cd ..'`
6. eseguire `'npm install'`
7. eseguire `'cd ..'`
8. eseguire `'docker-compose up -d'`
9. eseguire `'npm run init-db'` per inizializzare il database generando il calendario e lo storico di 15 partite di campionato già giocate.

Una volta che le seguenti istruzioni sono state effettuate, il software è correttamente installato e pronto per la messa in funzione.

7.2 Messa in funzione

1. dalla cartella backend eseguire `'npm start'`
2. dalla cartella frontend eseguire `'npm run dev -- -host'` in modo che sia raggiungibile anche da telefono.
3. collegarsi all'indirizzo `'http://localhost:5173/'`

8 Conclusioni

Tenendo conto che è un elaborato svolto in singolo e non in gruppo sono abbastanza soddisfatto del risultato finale, ci sarebbero state altre funzionalità che si potevano aggiungere ma che per mancanza di tempo non ho implementato, come ad esempio il requisito opzionale dell'elezione del miglior giocatore della partita, oppure ad esempio far inserire all'admin le formazioni della partita piuttosto che generare le formazioni di tutte le partite con lo script di inizializzazione del database.

Concludo mostrando un elenco di alcune immagini del risultato finale:

The screenshot displays the 'SERIE ASW' website interface. At the top, there are navigation tabs: 'CALENDARIO', 'CLASSIFICA', 'CLUB', and 'STATISTICHE'. The main content area is divided into two sections. The first section, titled 'GIORNATA 15 - 15 DIC 2025', shows six match results in a grid. The second section, titled 'CLASSIFICA', shows a table of the league standings. Below the table, there is a 'CLUB' section with a 'VEDI TUTTI' link and five club logos: AC Milan, Inter, Fiorentina, Juventus, and Como.

GIORNATA 15 - 15 DIC 2025		GIORNATA 15 - 14 DIC 2025		GIORNATA 15 - 14 DIC 2025		GIORNATA 15 - 14 DIC 2025		GIORNATA 15 - 14 DIC 2025		GIORNATA 15 - 14 DIC 2025	
1	Parma	3	Juventus	3	AC Milan	1	Lecce	3	Inter	1	Hellas Verona
0	Pisa	1	Udinese	2	Cremonese	1	Sassuolo	4	Atalanta	2	Bologna
0											

#	SQUADRA	PG	V	N	P	GF	GS	DR	PT	ULTIME 5
1	Como	15	9	2	4	35	31	4	29	● ● ● ● ●
2	Parma	15	9	2	4	35	26	9	29	● ● ● ● ●
3	Genoa	15	7	5	3	38	32	6	26	● ● ● ● ●
4	Inter	15	8	1	6	33	29	4	25	● ● ● ● ●
5	Torino	15	7	4	4	31	18	13	25	● ● ● ● ●

CLUB [VEDI TUTTI](#)

AC MILAN INTER FIORENTINA JUVENTUS COMO

Figure 9: Figura che mostra la pagina Home.

SERIE ASW

CALENDARIOCLASSIFICACLUSTATISTICHE



Classifica Serie A

#	SQUADRA	PG	V	N	P	GF	GS	DR	PT	ULTIME 5
1	 Como	15	9	2	4	35	31	4	29	    
2	 Parma	15	9	2	4	35	26	9	29	    
3	 Genoa	15	7	5	3	38	32	6	29	    
4	 Inter	15	8	1	6	33	29	4	25	    
5	 Torino	15	7	4	4	31	18	13	25	    
6	 Atalanta	15	7	2	6	33	36	-3	23	    
7	 Fiorentina	16	7	2	7	33	33	0	23	    
8	 AC Milan	15	6	5	4	26	20	6	23	    
9	 Hellas Verona	15	6	4	5	31	30	1	22	    
10	 Lecce	15	6	3	6	39	37	2	21	    
11	 Lazio	15	5	5	5	35	36	-1	20	    
12	 Udinese	15	6	2	7	25	28	-3	20	    
13	 Napoli	15	5	4	6	27	26	1	19	    

Figure 10: Figura che mostra lap pagina della classifica.

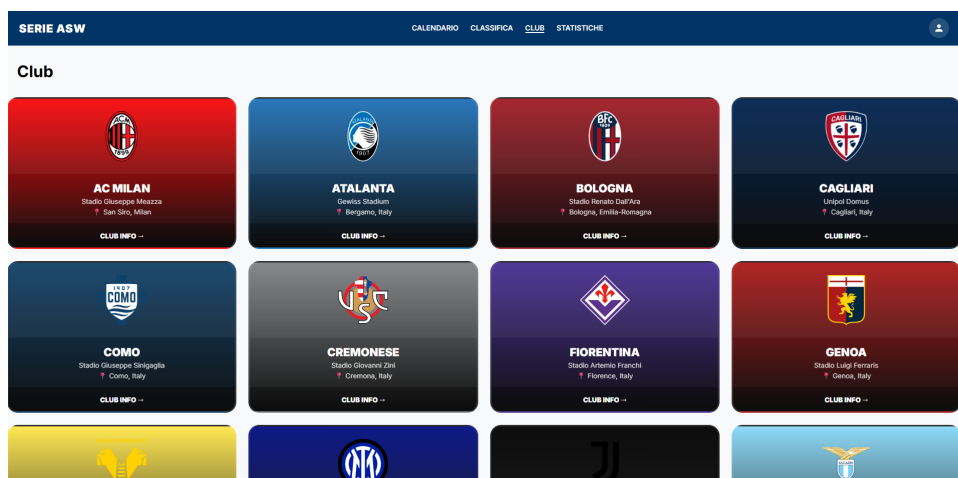


Figure 11: Figura che mostra la pagina delle squadre.

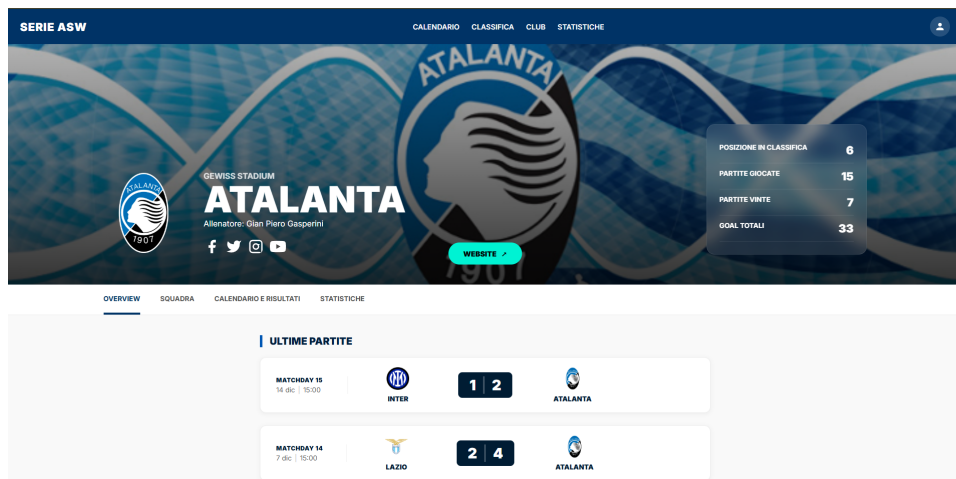


Figure 12: Figura che mostra la pagina di dettaglio della squadra.



Figure 13: Figura che mostra la pagina delle statistiche.

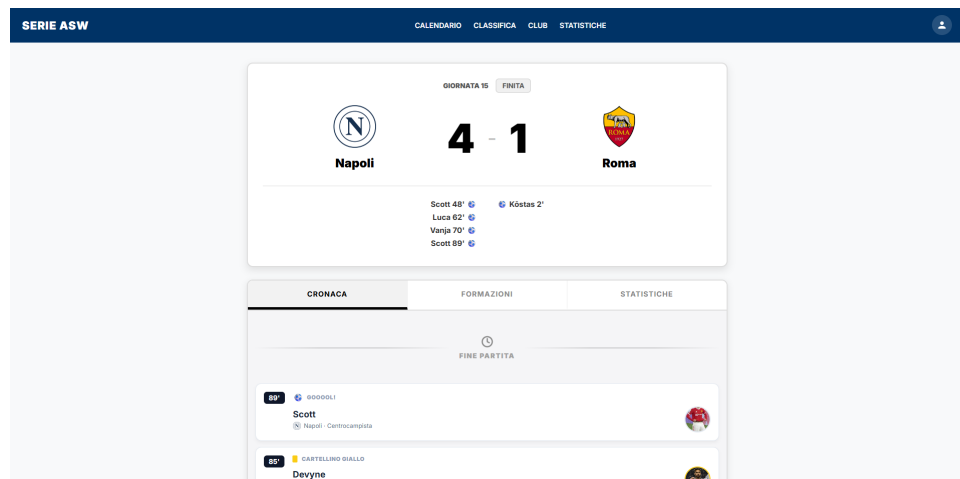


Figure 14: Figura che mostra la pagina della partita.

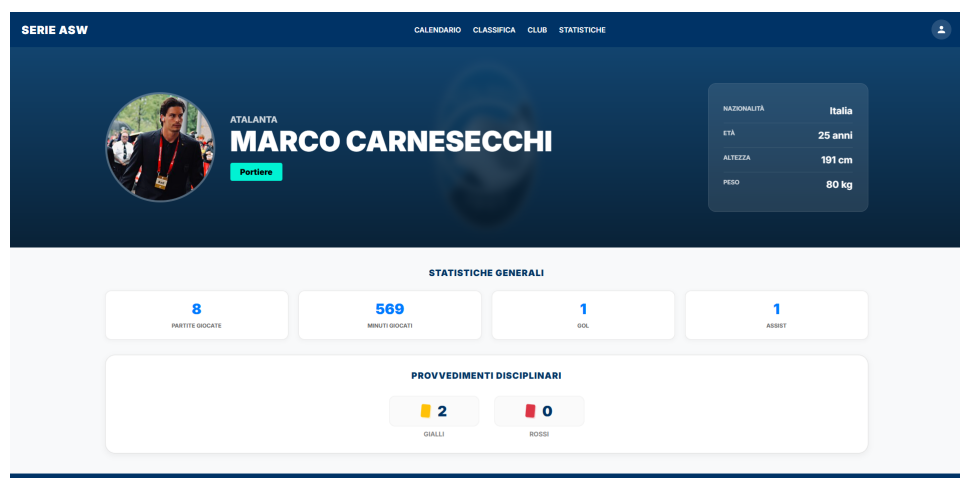


Figure 15: Figura che mostra la pagina del giocatore.

SERIE ASW

GIORNATA 15 - 14 DIC 2025

1



Lecce

3

1



Sassuolo

4

CLASSIFICA

#	SQUADRA	PG	V	N	P	GF
1	 Como	15	9	2	4	35
2	 Parma	15	9	2	4	35
3	 Genoa	15	7	5	3	38
4	 Inter	15	8	1	6	33
5	 Torino	15	7	4	4	31

Figure 16: Figura che mostra la pagina home da telefono.



Figure 17: Figura che mostra la pagina della partita da telefono.

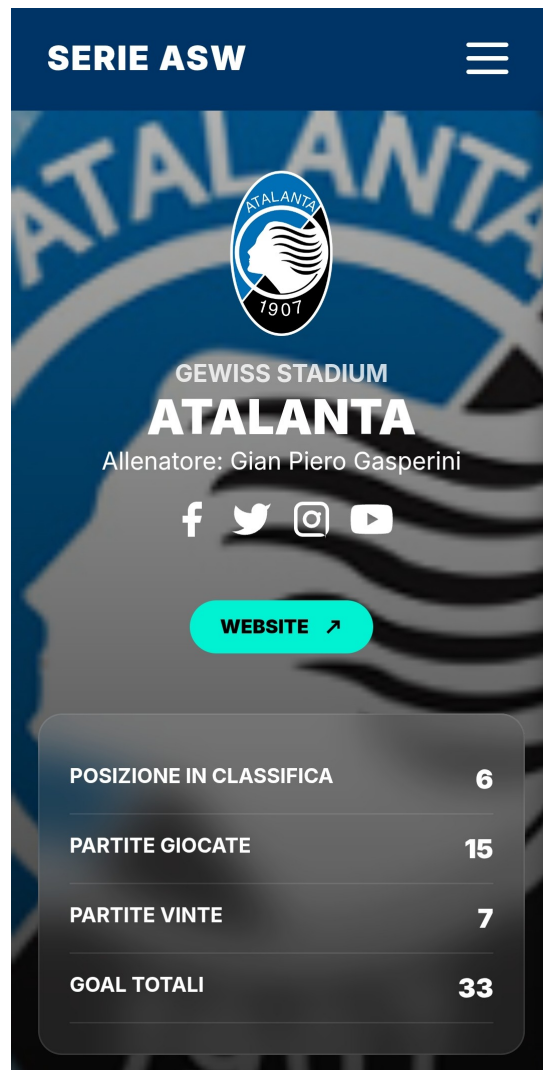


Figure 18: Figura che mostra la pagina della squadra da telefono.